

3 Use Case Description

Important: The following use case descriptions include detailed suggestions regarding technical implementation details, such as data structures (database tables, variable names) or user interface design (button labels, display texts).

Please consider all these details as one possible solution proposal from our side, not as a rigid specification. We explicitly encourage you to choose a different technical approach if it is smarter, more efficient, or more native to your platform.

The critical requirement for us is that the final implementation correctly and completely covers the described use case with all its functional steps and results.

This means: The “What” (business process) is fixed, the “How” (technical implementation) is at your discretion.

3.1 AI-powered Machine Alarm Use Case

3.1.1 High-Level Overview

The goal of this workflow is to inform a technician about a machine error and guide through the troubleshooting, utilizing AI assistance for fast resolution, and additionally enforcing a mandatory supervisor quality-release (Four-Eyes Principle).



3.1.2 Step-by-Step Description

Step 1: Automated Trigger and Notification to two operators per line

- The platform receives a live XML telegram (see 3.1.3 Mock Data)
- The system parses the XML and evaluates the business logic (see 3.1.4 Business Logic). If the criteria for task creation are met, the platform's internal data about user, roles and skills is queried, and a task created.
- The according technicians receive a notification on their mobile device: "Error at Line: \${v_lineNo}, Station: \${v_statNo}, Process: \${v_processName}".

Step 2: Task Assignment

- Two action buttons are presented: "Accept" and "Decline".
 - If the first Operator taps "Accept": The task is instantly assigned to this user, removed from other operators' device, and the screen transitions to the troubleshooting view.
 - If the Operator taps „Decline”: The task disappears from his device.
 - If a task is declined by both operators, the notification is resent to both again. After the notification has been resend the second time and still no operator accepted the task, the system bypasses standard technicians and escalates the notification directly to the active Supervisor.

Step 3: Troubleshooting & AI Assistance

- The task assignee walks to the physical machine and inspects the issue. The "Select Solution" screen is already open on their device, with a dynamic dropdown list prompting to select how the issue was resolved.
 - If \${v_errorNo} = "50" (Barcode Error): The dropdown displays:
 - "Cleaned sensor lens / camera"
 - "Replaced barcode scanner hardware"
 - "Adjusted scanner position / angle"
 - "Other"
 - If \${v_errorNo} = "70" (Transport Belt Error): The dropdown displays:
 - "Removed physical obstruction from belt"
 - "Replaced transport belt motor"
 - "Adjusted belt tension"
 - "Other"
- An AI assist button is available. Tapping this button opens an on-demand AI Chat window. The AI is fed automatically with contextual data (the parsed XML error metadata, relevant machine manuals from the plant's documentation system and historical resolution logs from DB_Shiftbook – see 3.1.3 Mock Data). Based on this context, the AI provides step-by-step troubleshooting guidance. The operator can ask follow-up questions, request detailed work steps, or ask for direct links to the exact page of the machine manual.
- Once the Operator selects one of the dropdown options, the "Next" button needs to be clicked to proceed with the next step.

Step 4: Evidence, Comment & Conditional Task Closure

- A camera interface prompting the operator to take a live photo of the repaired machine part.
- Once the photo is captured, the app transitions to a text-field screen with the prompt “Add a comment (optional)” where the operator can optionally type in a comment.
- Below the text-field, a button is presented:
 - If `#{v_errorNo} = "70"`: The button displays "Close Task". Tapping it finalizes and closes the task. Proceed with step 6.
 - If `#{v_errorNo} = "50" AND the current user has the role = "Supervisor"`: The button displays "Close Task". Tapping it finalizes and closes the task. Proceed with step 6.
 - If `#{v_errorNo} = "50" AND the current user has the role = "Technician"`: The button displays "Request 4-Eyes Release". Tapping it escalates the task to the current supervisor. Proceed with step 5.

Step 5: Supervisor Approval

- The supervisor receives the notification with following details:
 - Error Details: `#{v_errorNo} | #{v_errorText} | #{v_lineNo} | #{v_statNo}`
 - Operator's Input: Selected Solution, Live Photo Proof, and Optional Comments.
- After the supervisor verified the resolution, the button “Approve & Close” needs to be tapped. Tapping it finalizes and closes the task.

Step 6: Storing the Finalized Record

- Once the task is closed, the platform compiles all transaction details: operator ID, error code, solution selected, precision timestamp, photo proof, operator comment, and supervisor ID (if escalated). This finalized record is saved as a new row in the DB_Shiftbook database (see 3.1.3 Mock Data).

3.1.3 Mock Data

3.1.3.1 XML Payload Structure

The XML structure provided below serves as a template to simulate a machine alarm via an API client. Please set up a corresponding API endpoint capable of receiving this data format. During the presentation, we will be changing the values within the XML payload live to test the dynamic behaviour of your system.

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
  <header eventId="12345" version="2.3" eventName="plcError" eventSwitch="-1"
contentType="3">
    <location lineNo="31" statNo="20" statIdx="1" fuNo="1" processName="Paste Printer"
processNo="120" application="IPC" />
  </header>
  <event>
    <plcError typeNo="0123456789" typeVar="0001" errorNo="50" errorText="Barcode not
readable" errorType="1" errorState="0" operationMode="1" chainNo="999" />
  </event>
  <body/>
</root>
```

3.1.3.2 Historical Shiftbook Database DB_Shiftbook

This database serves as logbook for all resolved machine alarms. While the system automatically writes a new entry every time a task is completed, the database provides the essential training data for the AI Assistant.

Please note, the ID values provided in the example are for illustrative purposes only. You can adapt them so the logic and technical requirements best suits your platform.

That's the structure:

log_id	timestamp_resolved	line_no	error_no	resolved_by_user_id	solution_selected	operator_comment	supervisor_user_id
5501	2026-07-07 09:15:21	10	70	101	Replaced transport belt motor	NULL	NULL
5502	2026-07-07 10:05:45	31	50	102	Cleaned sensor lens / camera	"Sensor was very dusty"	201
5503	2026-07-06 14:20:11	31	50	103	Adjusted scanner position / angle	"Bracket was loose"	201

3.1.3.3 Internal User, Role and Skill Management

For the demonstration, the platform must be configured with user profiles. This data is managed internally within the platform and simulates the on-site team structure, dictating who receives which task notification based on their assigned line, role, and skills.

display_name	role	assigned_lineNo	skill_set
John Doe	Technician	10	Transport Belt Maintenance
Jane Smith	Technician	10, 31	Barcode Maintenance
Alice Wonder	Technician	31	Transport Belt Maintenance, Barcode Maintenance
Bob Builder	Supervisor	10, 31	Transport Belt Maintenance, Barcode Maintenance

3.1.4 Business Logic

A. LineNo

- Purpose: Identifies the physical location of the machine within the plant and restricts the task distribution to the correct team.
- Logic:
 - If lineNo="31": Route only to the 2 technicians assigned to Line 31.
 - If lineNo="10": Route only to the 2 technicians assigned to Line 10.

B. StatNo & processName

- Purpose: Identifies the specific machine station where the error occurred.
- Logic:
 - This field is kept static for the demonstration.
 - The platform must parse and display this static string on the user interface to show the operator the exact machine name.

C. ErrorNo

- Purpose: Dictates who gets the task and how the task is closed.
- Logic:
 - If errorNo = "50" (Barcode error): Send to all active operators assigned to the according line. A mandatory "Four-Eyes" release by the active supervisor is required to close the task.

- If errorNo = "70" (Transport belt error): Route to technicians assigned to the skill "Transport Belt Maintenance". Allow the technician to close the task directly.

D. ErrorText

- Purpose: Provides a human-readable description of the error to be displayed on the UI and analyzed by the AI Assistant.
- Logic:
The text dynamically corresponds to the event.errorState:
 - If event.errorState = "0" (Error occurred): The text must display "Barcode not readable" (for Error 50) or "Transport belt error" (for Error 70).
 - If event.errorState = "1" (Error resolved): The text dynamically updates to "acknowledge".

E. ErrorType

- Purpose: Classifies the severity of the issue to define the operator's response urgency.
- Logic:
 - If errorType="1" (Error): Issues that stop the automatic mode of the machine immediately. The operator must support directly. A high priority task should be created.
 - If errorType="2" (Warning): Issues that do not stop the automatic mode immediately. The operator must come soon to resolve it, but no immediate line stoppage occurs. A low priority task should be created.

F. ErrorState

- Purpose: Determines if the error is active or has already been resolved/cleared by the machine itself.
- Logic:
 - If errorState = "0": The error is active. The platform must trigger a task and send push notifications.
 - If errorState = "1": The error no longer exists on the physical machine (e.g., resolved via machine reset). The platform must automatically archive/close the active task in the background to prevent "ghost alarms" (unnecessary walks to the machine).

G. OperationMode

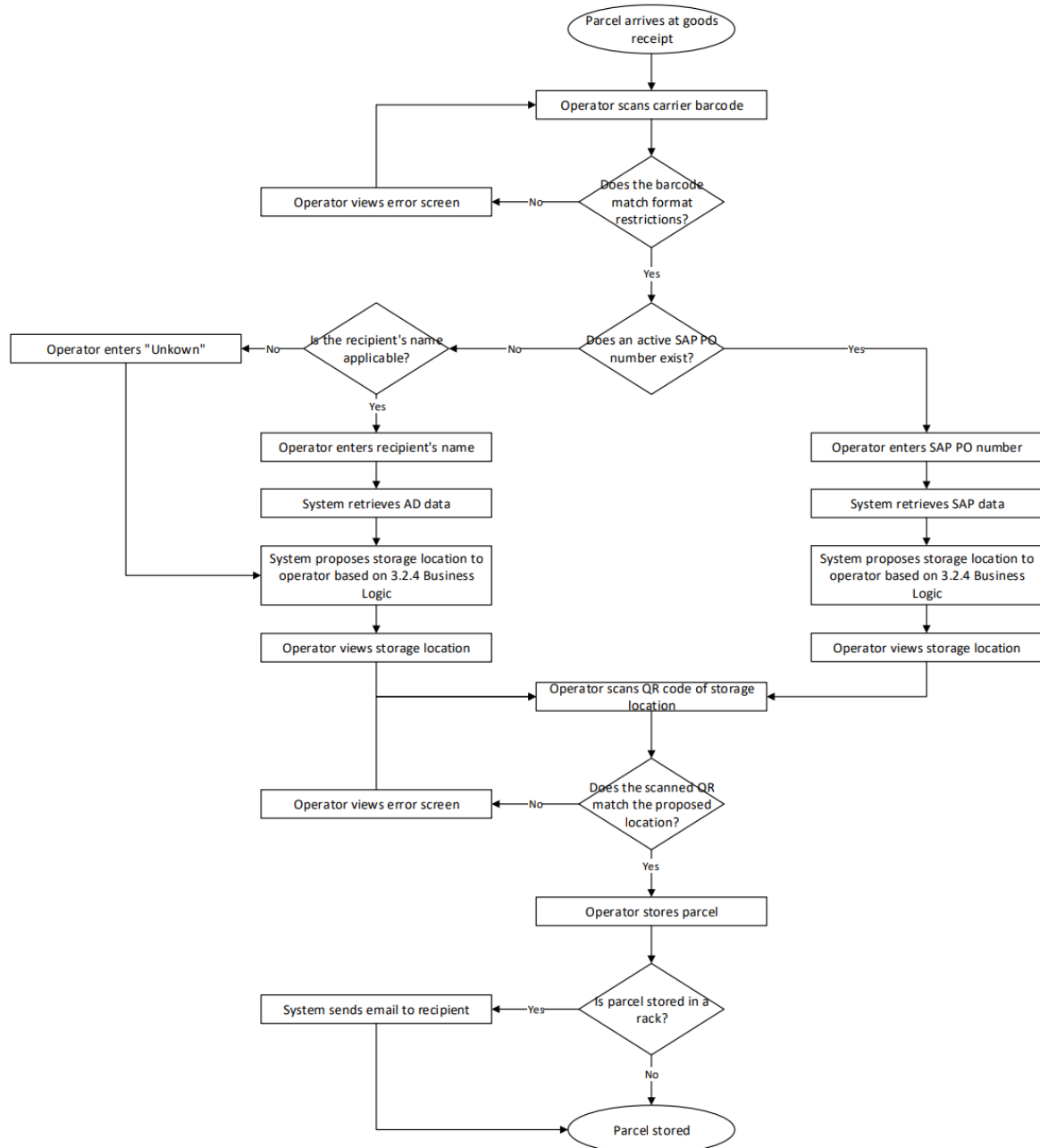
- Purpose: Determines if the machine is actively running in production or being worked on manually.
- Logic:
 - If operationMode = "1" (Automatic Mode): The platform must create a task and send a notification.
 - If operationMode = "3" (Manual Mode): The alarm is ignored (no task is created), because a technician is already at the machine.

3.2 Inbound and Parcel Status Use Case

3.2.1 High-Level Overview

The goal is to digitize and error-proof the goods receipt process (App 1) and monitor the parcel status in real-time (App 2/ Dashboard).

- App 1 – Inbound Registration:



- App 2/ Dashboard – Parcel Status: Acts as a central tracking hub. It allows e.g. shift leaders or recipients to monitor and filter the real-time status of all inbound shipments as well as internal deliveries.

3.2.2 Step-by-Step Description

Note: We will provide all required location QR-Codes and carrier labels during the session. There is no need for you to prepare any codes or labels in advance.

3.2.2.1 Inbound Registration

Step 1: Tracking number scanning and format validation

- The operator picks up a newly arrived parcel at the goods receipt station, opens the app and views the start screen.
- The screen displays a header “Please scan the tracking number”, along with a list of supported carriers (“DHL, UPS, GLS, Amazon”) and an option to start the scanning.
- The operator scans the carrier’s shipping label on the parcel.
 - If the format is invalid (*see 3.2.4 Business Logic*): The screen turns red and displays the error message: „Invalid Format! Please scan a valid carrier label.“. The operator must scan again until the format is valid.
 - If the format is valid: The screen turns green and displays a confirmation: “\${v_tracking_id} from \${v_carrier} has been successfully registered”. By clicking “Next”, the next screen opens.

Step 2: Recipient Identification & Location Proposal

- On the next screen, the app is asking if an active SAP PO number does exist. The operator checks the parcel and can choose between a “Yes” and “No” button.
- If the operator taps “Yes”:
 - An input field appears prompting for the 10-digit SAP PO number. The operator can only submit the number by tapping “Next”, if 10 digits has been entered.
 - After the number is entered, the system retrieves the associated order details. Crucially, the system determines the best storage location based on data availability (*see 3.2.4 Business Logic*).
 - The screen displays the retrieved information and the system-proposed storage location:
 - Recipient: \${v_recipient_name} (*This may be a person's name, a department name, or "N/A"*)
 - Department: \${v_department} (*May be blank*)
 - Go To Location: \${v_proposed_location} (*e.g., "RACK-A-04" or "TROLLEY-01"*)
 - Below, a “Scan Location” button is presented.
- If the operator taps “No”:
 - A text field appears: "Enter recipient's full name from package label".
 - The operator types the name. If no name is visible on the package, they must enter "Unknown". The “Next” button is only enabled after text is entered.
 - The system then proposes the best storage location (*see 3.2.4 Business Logic*), and the screen displays the information:
 - Recipient: \${v_recipient_name}
 - Department: \${v_department}
 - Go To Location: \${v_proposed_location}
 - Below, a “Scan Location” button is presented.

Step 3: Location Verification

- The operator walks the parcel to the location shown on the screen and taps the "Scan Location" button. They then scan the QR-Code on the physical rack or trolley.
 - If the scan is incorrect: The screen turns red and displays the message: "Wrong location! Expected: \${v_proposed_location}. Scanned: \${v_actual_location}. Please scan the correct QR-Code." The process is blocked until the proposed location is scanned.
 - If the scan is correct: The screen turns green and displays: "Location verified! You can now place the parcel in \${v_proposed_location}." A "Next" button is presented.

Step 4: Parcel Storing and Process Completion

- The operator physically places the parcel in the verified location.
- A final completion screen with a "Close App" button appears: "Process completed successfully. You can now close the app."
- The system finalizes the booking, logs all data, and triggers the email notification if the storage location was a rack.

3.2.2.2 Parcel Status

Concept:

This serves as the central, real-time information hub for all tracked parcels within the plant. Its purpose is to provide supervisors, shift leaders, and employees with a self-service tool to quickly find the status and location of any registered parcel or internal transit item. Here, the format for the view is flexible (e.g. mobile dashboard, dedicated search app, etc.). You should choose the format that best demonstrates your platform's strength.

Core Functional Requirements:

- The view must be a read-only interface that queries the unified transaction table `DB_Parcel_Platform.tbl_parcel`s (see 3.2.3 Mock Data) in real-time. The database does not only store the inbound registration from App 1 but also includes data from other plant logistics system e.g. milkruns. Please note that you don't need to design additional processes to populate data from other plant logistics systems. Fictional dummy data is completely sufficient to make App 2 functional for the demo.
- The user interface must provide a clear and intuitive way to filter and search the data.
- At a minimum, the user must be able to filter by:
 - Carrier
 - Date
 - Recipient Name
 - Status (e.g., STORED, IN_TRANSIT, DELIVERED)
- The display should focus on clarity and essential information. The following variables must be clearly visible for each record:
 - Tracking ID (`${v_tracking_id}`)
 - Carrier (`${v_carrier}`)
 - Current Status (`${v_status}`)
 - Storage Location (`${v_actual_location}`)
 - Recipient Name (`${v_recipient_name}`)
 - Date (`${v_timestamp_last_event}`)

3.2.3 Mock Data

3.2.3.1 Internal Platform Databases (DB_Parcel_Platform)

Tabel A: tbl_parcel

This table stores a record for every item tracked by the system, whether it is an inbound parcel (logged by App 1) or an internal transit item. That's the schema:

- tracking_id (VARCHAR, Primary Key): The unique carrier tracking ID or internal tracking number.
- carrier (VARCHAR): Detected provider (DHL, UPS, GLS, Amazon) or internal source.
- sap_po_number (VARCHAR, Nullable): Entered SAP Purchase Order.
- recipient_name (VARCHAR, Nullable): Name of the recipient if applicable.
- proposed_location (VARCHAR, Nullable): Target location calculated by the system.
- actual_location (VARCHAR): QR-Code scanned physically at the rack/trolley.
- status (VARCHAR): Current processing state (STORED, IN_TRANSIT, DELIVERED).
- timestamp_last_event (DATETIME): The timestamp of the last event that set the status and location.

This table starts empty at the beginning of the demo, but here is what it looks like after a few successful scans:

tracking_id	carrier	sap_po_number	recipient_name	proposed_location	actual_location	status	timestamp_last_event
JD0123456789012345	DHL	4500987654	John Doe	RACK-A-04	RACK-A-04	STORED	2026-07-06 08:12:00
1Z999AA10123456784	UPS	NULL	Alice Wonder	TROLLEY-01	TROLLEY-01	STORED	2026-07-06 09:30:15
MR-2026-07-08-001	Internal Milkrun	NULL	NULL	NULL	LINE_B_STAGING	IN_TRANSIT	2026-07-06 10:05:00

Tabel B: tbl_storage_locations

This table acts as the digital map of all physical storage locations. That's the schema:

- location_id (VARCHAR, Primary Key): The physical QR-Code value.
- location_type (VARCHAR): Category of location (RACK or TROLLEY).
- assigned_department (VARCHAR, Nullable): Restricts rack usage to specific departments.
- is_occupied (BOOLEAN): Status flag (TRUE if a parcel is currently stored there, FALSE if vacant).

location_id	location_type	assigned_department	is_occupied
RACK-A-04	RACK	MOE/LOG-A	TRUE
RACK-A-05	RACK	MOE/LOG-A	FALSE (Vacant, ready to be proposed)
RACK-C-12	RACK	NULL	TRUE
RACK-C-13	RACK	NULL	FALSE (Vacant general rack)
TROLLEY-01	TROLLEY	NULL	FALSE (Always available for transit)

3.2.3.2 *Simulated External Systems*

System A: SAP_ERP

Simulates the Bosch SAP backend. When App 1 queries an SAP PO number, this system returns the recipient metadata.

sap_po_number	recipient_name	department	order_status
4500987654	John Doe	MOE/LOG-A	ACTIVE
4500987655	Bob Builder	MOE/ENG-2	ACTIVE
4500111222	Sarah Connor	MOE/MFG-P	COMPLETED

System B: Bosch_Active_Directory

Simulates the central active directory. When App 1 needs to send an email, it queries this table with the recipient's name to find their exact corporate email.

nt_user_id	recipient_name	email_address	department
DOE2AN	John Doe	john.doe@bosch.com	MOE/LOG-A
BUI4AN	Bob Builder	bob.builder@bosch.com	MOE/ENG-2
CON1AN	Sarah Connor	sarah.connor@bosch.com	MOE/MFG-P
WON5AN	Alice Wonderland	alice.w@bosch.com	MOE/LOG-A

3.2.4 Business Logic

A. Tracking format validation

The scanner field in step 1 must restrict inputs using these exact expressions:

- DHL: `^JD[0-9]{16}$` (Starts with JD followed by 16 digits)
- UPS: `^1Z[A-Z0-9]{16}$` (Starts with 1Z followed by 16 alphanumeric characters)
- GLS: `^[0-9]{12}$` (Exactly 12 digits)
- Amazon: `^TBA[0-9]{12}$` (Starts with TBA followed by 12 digits)

B. Recipient & Location Logic

The system logic for proposing a storage location is a prioritized cascade that applies to both SAP and non-SAP paths.

1. Get recipient name and department:

- If SAP PO exists: The system queries SAP_ERP to get `#{v_recipient_name}` (which could be a person, a department, or NULL) and `#{v_department}` (could be NULL).
- If no SAP PO exists: The operator must manually enter `#{v_recipient_name}`. The system then queries the Bosch_Active_Directory using `#{v_recipient_name}`. This query should attempt to return the employee's department, which is then stored in `#{v_department}`. If the recipient's name is unknown, the system sets `#{v_recipient_name}` and `#{v_department}` to NULL and no query to the Bosch_Active_Directory is needed.

2. Location proposal cascade

- Priority 1 (Department rack): The system first checks if a `#{v_department}` value was successfully retrieved (either from SAP or Active Directory). IF YES, it queries DB_Parcel_Platform.tbl_storage_locations for a vacant location (`is_occupied = FALSE`) where `assigned_department` matches `#{v_department}`.
- Priority 2 (General rack - Fallback): IF no department was found, OR IF no free department-specific rack is available, the system queries for a vacant general rack (where `assigned_department` is NULL).
- Priority 3 (Transit trolley - Final fallback): IF no general rack is available, the system proposes the default transit trolley (e.g., TROLLEY-01).
- The first location found in this sequence is set as `#{v_proposed_location}`.

C. Location Verification

- IF (`#{v_actual_location} == #{v_proposed_location}`) THEN allow the booking to proceed.
- IF (`#{v_actual_location} != #{v_proposed_location}`) THEN block the transaction and display the error screen

D. Finalization & Email Notification

Database Write:

- An INSERT statement is executed on DB_Parcel_Platform.tbl_parcel to save all collected session variables (`#{v_tracking_id}`, `#{v_recipient_name}`, `#{v_actual_location}`, etc.).

Database Update:

- An UPDATE statement is executed on DB_Parcel_Platform.tbl_storage_locations, setting `is_occupied` to TRUE for the location `#{v_proposed_location}`.

Email Trigger Logic:

- IF (\${v_proposed_location} CONTAINS "RACK") AND (\${v_recipient_email} IS NOT NULL) THEN trigger an API call to the SMTP Gateway to dispatch the notification mail.
 - Subject: Your parcel is ready for pickup at Goods Receipt
 - Body: Your parcel with the tracking ID \${v_tracking_id} from carrier \${v_carrier} has been safely stored and is ready for pickup. Pickup Location: \${v_actual_location}
- ELSE (if location is "TROLLEY" or email is NULL) skip the email dispatch.